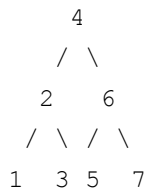


Inorder Traversal — Complete Guide

1. What is it?

Inorder traversal visits a binary tree in the order: **Left → Root → Right**.

For a **Binary Search Tree (BST)**, this produces nodes in **sorted (ascending) order**. That single fact is the reason inorder is special — for a plain binary tree, inorder is “just another traversal,” but for a BST it’s a sorted-sequence generator.



Inorder: 1 2 3 4 5 6 7 (sorted!)

2. When & Why is it used?

Used whenever you need **sorted order from a BST**, or need to process nodes such that “everything smaller comes before everything bigger.”

Why: Because inorder is the *only* one of the three DFS traversals (pre/in/post) that respects BST ordering. It converts a tree structure into a linear sorted sequence in $O(n)$, without sorting.

3. Where is it used? (real applications)

- Validate whether a tree is a valid BST
- Find kth smallest / kth largest element in a BST
- Convert BST → sorted array / sorted doubly linked list
- Find inorder successor/predecessor of a node
- Check if two BSTs have the same elements

- Range sum / range queries on BST
- Balancing a BST (get sorted array, rebuild balanced tree)
- Expression tree evaluation (for certain expression forms)
- Two-sum in a BST (using two pointers via inorder iterator, or two stacks front/back)

4. How do you identify it's an inorder problem?

► Pattern triggers (the "2-minute signal")

If the question mentions **any** of these, think inorder immediately:

Signal phrase in problem	Why it screams "inorder"
"Binary Search Tree" (BST specifically, not just binary tree)	BST + order → inorder gives sorted sequence for free
"kth smallest" / "kth largest"	Sorted order needed → inorder + counter
"sorted order" / "ascending order" / "increasing order"	Direct definition of inorder
"validate BST" / "is this a valid BST"	Inorder must be strictly increasing
"convert BST to sorted array/list"	Literally what inorder does
"successor" / "predecessor" in a BST	Next/prev element in sorted sequence
"two elements sum to target" in a BST	BST-two-sum via inorder iterators (like sorted-array two pointer)
"closest value" in BST	Needs sorted traversal / narrowing
Mentions " in-place " conversion of BST + "O(1) space"	→ Morris Inorder Traversal specifically

🎯 One-line heuristic

"Binary Search Tree" + "order/sorted/kth/rank" in the same sentence = inorder.

If it's just a **binary tree** (not BST) and the question is about structure/depth/paths — it's NOT an inorder problem, it's usually pre/post-order or BFS.

Can you tell from constraints alone?

Yes, somewhat:

- **Node count up to 10^4 – 10^5 , values distinct, "BST" mentioned** → strongly suggests inorder-based solution is intended ($O(n)$ time).
- **"Follow-up: $O(1)$ space" or "no recursion/no extra data structure allowed"** → signals they want **Morris Inorder Traversal** (constraints are testing whether you know the $O(1)$ -space variant).
- **"Follow-up: what if tree is modified frequently, kth smallest is called repeatedly"** → signals they want an **iterative controlled inorder with a stack you can pause/resume**, not full traversal each time.
- **$k \leq n$ given explicitly** → confirms kth-smallest-style inorder with early termination (don't traverse the whole tree, stop once you've seen k nodes).

5. Types / Variants

Type	Space	Notes
Recursive	$O(h)$ call stack	Simplest, most natural to write
Iterative (explicit stack)	$O(h)$	Needed when recursion depth is a concern or interviewer asks "do it without recursion"
Morris Traversal (threaded)	$O(1)$	Uses temporary links (threads) via right pointers of predecessor; no stack, no recursion
Iterative with early stop	$O(h)$	For kth-smallest — stop as soon as counter hits k , don't traverse rest of tree

6. Rationale / Intuition

Think of a BST's structure as encoding an ordering: everything in the left subtree $<$ root $<$ everything in the right subtree. That's true **recursively** at every node.

Inorder traversal is literally the recursive definition of "in sorted order": *first exhaust everything smaller (left subtree), then take yourself, then exhaust everything bigger (right*

subtree). It's the same idea as an in-order arithmetic expression ($a + b$, operator between operands) — hence the name “in-order.”

Iterative inorder simulates recursion using an explicit stack: keep pushing left children until null, then pop (that's the “visit” point), then move right.

Morris traversal avoids the stack entirely by **temporarily rewiring** the tree: for each node, find its inorder predecessor (rightmost node in left subtree) and make its right pointer point back to the current node — a temporary thread. This lets you “return” to the current node after finishing the left subtree without a stack, then you remove the thread to restore the tree.

7. Time & Space Complexity

Traversal type	Time	Space
Recursive	$O(n)$	$O(h)$ — h = tree height ($O(\log n)$ balanced, $O(n)$ skewed)
Iterative (stack)	$O(n)$	$O(h)$
Morris	$O(n)$	$O(1)$ (amortized — each edge visited at most twice)

n = number of nodes, h = height of tree.

8. Comparison with other traversals

Traversal	Order	Used for
Preorder (Root, Left, Right)	Root first	Cloning/serializing a tree, prefix expression, tree-to-string, building tree from scratch
Inorder (Left, Root, Right)	Sorted (for BST)	Sorted output, BST validation, kth smallest
Postorder (Left, Right, Root)	Children before parent	Deleting a tree, evaluating expression trees, computing subtree properties (height, diameter, sum) bottom-up
Level order	Level by	Shortest path in unweighted tree, level-based aggregation,

(BFS)	level	printing tree by levels
-------	-------	-------------------------

Quick mnemonic: **Pre**order = process before children (top-down build). **Post**order = process after children (bottom-up aggregate). **In**order = process between children (sorted, BST-specific).

9. Reusable Java Templates

a) Recursive

```
class Solution {
    List<Integer> result = new ArrayList<>();

    public List<Integer> inorderTraversal(TreeNode root) {
        inorder(root);
        return result;
    }

    private void inorder(TreeNode node) {
        if (node == null) return;
        inorder(node.left);
        result.add(node.val);    // "visit" point
        inorder(node.right);
    }
}
```

b) Iterative (explicit stack) — most reusable for interviews

```
public List<Integer> inorderTraversal(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        // go as far left as possible
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        // visit
        curr = stack.pop();
        result.add(curr.val);
    }
}
```



```

        // move to right subtree
        curr = curr.right;
    }
    return result;
}

```

c) Iterative with early termination — kth smallest pattern

```

public int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        if (--k == 0) return curr.val;    // early stop, no need to finish travers
        curr = curr.right;
    }
    throw new IllegalArgumentException("k out of bounds");
}

```

d) Morris Traversal — O(1) space

```

public List<Integer> morrisInorder(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    TreeNode curr = root;

    while (curr != null) {
        if (curr.left == null) {
            result.add(curr.val);
            curr = curr.right;
        } else {
            // find inorder predecessor of curr
            TreeNode pred = curr.left;
            while (pred.right != null && pred.right != curr) {
                pred = pred.right;
            }
            if (pred.right == null) {
                // create thread, go left
                pred.right = curr;
                curr = curr.left;
            }
        }
    }
}

```

```

        } else {
            // thread already exists -> left subtree done, remove thread
            pred.right = null;
            result.add(curr.val);
            curr = curr.right;
        }
    }
}
return result;
}

```

e) Validate BST using inorder (bounds check, no extra list)

```

public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

private boolean validate(TreeNode node, Integer lower, Integer upper) {
    if (node == null) return true;
    if (lower != null && node.val <= lower) return false;
    if (upper != null && node.val >= upper) return false;
    return validate(node.left, lower, node.val) &&
        validate(node.right, node.val, upper);
}

```

(This is the inorder-property applied without building a list — $O(1)$ extra space beyond recursion stack.)

10. Quick decision checklist (use this in the actual interview)

1. Is it a **BST**? → inorder is on the table.
2. Do they want **sorted order / kth element / validation**? → confirmed, use inorder.
3. Any **space constraint ($O(1)$)**? → use Morris.
4. Any **early-exit condition (kth smallest, closest value)**? → use iterative stack with early break, don't do full traversal + collect.
5. Otherwise → simple recursive inorder is the fastest to write and reason about.